Nom : Azarkan

Prénom : Ilyas

Projet JavaIV – 2025/2026

HELBMaGo

Professeur : Riggio Jonathan

Date : 10/08/2026

Table des matières

1. **Introduction**

2. **Fonctionnalités de base**
   - 2.1 Gestion des MaGos
   - 2.2 Gestion des messages
   - 2.3 Réaction des MaGos
   - 2.4 Interface graphique
   - 2.5 Fenêtre d'un contrôle d'un MaGo
   - 2.6 Sélection des messages
   - 2.7 Gestion du timer

3. **Analyse et application des Design Patterns**
   - 3.1 Architecture MVC
   - 3.2 Pattern Singleton
   - 3.3 Pattern Factory
   - 3.4 Pattern Strategy
   - 3.5 Pattern Observer
   - 3.6 Justification des choix

4. **Limitations**
   - 4.1 Gestion des messages
   - 4.2 Nombre de types de MaGos
   - 4.3 Gestions des erreurs

5. **Conclusion**

6. **Accès LLM**

1. Introduction

Le projet HELBMaGo consiste à développer une application Java orientée objet permettant de simuler une population de créatures appelées MaGos. Chaque MaGo possède un niveau de bonheur qui évolue au fil des messages affichés par l'application. Selon son type, un MaGo réagit différemment au contenu des messages, ce qui influence directement son état ainsi que sa représentation dans l'interface graphique.

L'application permet à l'utilisateur de visualiser les différents MaGos, de consulter les messages diffusés, de choisir une stratégie de sélection des messages et de modifier manuellement le niveau de bonheur d'un MaGo. Les messages sont renouvelés automatiquement grâce à un système de timer, mais l'utilisateur peut également passer au message suivant à tout moment.

D'un point de vue technique, le projet est basé sur une architecture MVC (Model – View – Controller) qui permet de séparer clairement les différentes responsabilités de l'application. Plusieurs design patterns ont également été utilisés afin de rendre le code plus modulaire, maintenable et facilement extensible, notamment les patterns Singleton, Factory, Strategy et Observer.

Une attention particulière a également été portée à la qualité du code. Les différentes classes sont documentées à l'aide de commentaires Javadoc, les constantes ont été centralisées dans une classe dédiée afin d'éviter les constantes magiques et des tests unitaires ont été réalisés.

Ce rapport présente tout d'abord les principales fonctionnalités offertes par l'application, aussi bien d'un point de vue fonctionnel que technique. Il décrit ensuite les différents design patterns mis en œuvre ainsi que leur rôle dans l'architecture du projet. Enfin, les principales limitations de l'application sont présentées avant de conclure sur les apprentissages réalisés au cours de ce projet.

2. Fonctionnalités de base

2.1 Gestion des MaGos

L'application repose sur une population de MaGos créée automatiquement au démarrage. Cette population est générée par la classe MaGoFactory, qui détermine aléatoirement le nombre de MaGos à créer dans une plage définie par les constantes de l'application.

Chaque MaGo appartient à l'un des deux types disponibles :

- Circle, représenté par un cercle ;
- Square, représenté par un carré.

Le type de chaque MaGo est déterminé à partir du timestamp courant, ce qui permet d'obtenir une population différente à chaque exécution de l'application. Chaque MaGo possède un niveau de bonheur initialisé à zéro et limité entre -100 et 100 afin de garantir la cohérence de son état.

Le comportement propre à chaque type est implémenté par héritage à partir de la classe abstraite MaGo, ce qui permet de spécialiser uniquement les réactions aux messages tout en mutualisant le reste du comportement.

2.2 Gestion des messages

Les messages affichés dans l'application sont chargés depuis un fichier texte au démarrage grâce à la classe MessageReader. Avant d'être utilisés, ils sont analysés par SpamValidator, qui élimine automatiquement les messages considérés comme du spam.

Un message est identifié comme un spam lorsqu'il contient un lien Internet commençant par http:// ou lorsqu'il ne contient aucune lettre. Les messages valides sont ensuite stockés dans une liste gérée par MessageManager.

À chaque affichage, le message sélectionné est retiré de la liste des messages disponibles afin d'éviter les répétitions. Lorsque tous les messages ont été utilisés, la liste est automatiquement reconstruite à partir des messages d'origine.

2.3 Réaction des MaGos

Chaque MaGo réagit différemment selon son type.

Les Circle analysent la ponctuation du message. Chaque point d'exclamation ou point d'interrogation augmente leur niveau de bonheur, tandis qu'un message ne contenant aucune de ces ponctuations entraîne une diminution de leur bonheur.

Les Square analysent quant à eux le rapport entre le nombre de voyelles et le nombre de consonnes présentes dans le message. Lorsque ce ratio dépasse un seuil prédéfini, leur bonheur augmente ; dans le cas contraire, il diminue.

Après chaque réaction, le niveau de bonheur est automatiquement limité entre les valeurs minimales et maximales autorisées.

2.4 Interface graphique

L'application dispose d'une interface graphique composée de trois parties principales.

Le panneau supérieur affiche le bonheur moyen de l'ensemble des MaGos ainsi qu'un indicateur de couleur permettant de visualiser rapidement leur état général.

Le panneau central présente le message courant, le temps restant avant l'affichage du prochain message ainsi que les différentes stratégies de sélection disponibles. L'utilisateur peut également afficher immédiatement le message suivant grâce au bouton dédié.

Enfin, le panneau inférieur représente graphiquement l'ensemble des MaGos. Chaque MaGo est dessiné selon son type (cercle ou carré) et sa couleur évolue automatiquement en fonction de son niveau de bonheur.

2.5 Fenêtre de contrôle d'un MaGo

L'utilisateur peut cliquer sur n'importe quel MaGo affiché dans le panneau inférieur afin d'ouvrir une fenêtre de contrôle.

Cette fenêtre affiche le niveau de bonheur actuel du MaGo et permet de saisir une nouvelle valeur comprise entre les limites autorisées. La validité de la saisie est contrôlée automatiquement et le bouton de confirmation n'est activé que lorsque la valeur introduite est correcte.

Une fois la modification validée, le bonheur du MaGo est mis à jour et l'ensemble de l'interface est automatiquement rafraîchi afin de refléter le nouvel état.

2.6 Sélection des messages

L'application propose plusieurs stratégies permettant de sélectionner le prochain message.

La stratégie Random choisit un message de manière totalement aléatoire.

La stratégie GoCircle sélectionne le message susceptible d'augmenter le plus le bonheur moyen des MaGos de type Circle.

La stratégie GoSquare applique le même principe pour les MaGos de type Square.

Enfin, la stratégie GoAny choisit aléatoirement l'un des deux types présents dans la population avant d'appliquer la stratégie d'optimisation correspondante.

2.7 Gestion du timer

L'affichage des messages est automatisé grâce à un timer. Toutes les cinq secondes, un nouveau message est sélectionné selon la stratégie actuellement choisie et les MaGos réagissent à son contenu.

Le timer est automatiquement réinitialisé lorsqu'un nouveau message est affiché manuellement par l'utilisateur. Il est également suspendu lors de l'ouverture de la fenêtre de contrôle d'un MaGo afin d'éviter qu'un changement de message ne survienne pendant la modification de son niveau de bonheur.

3. Analyse et application des Design Patterns

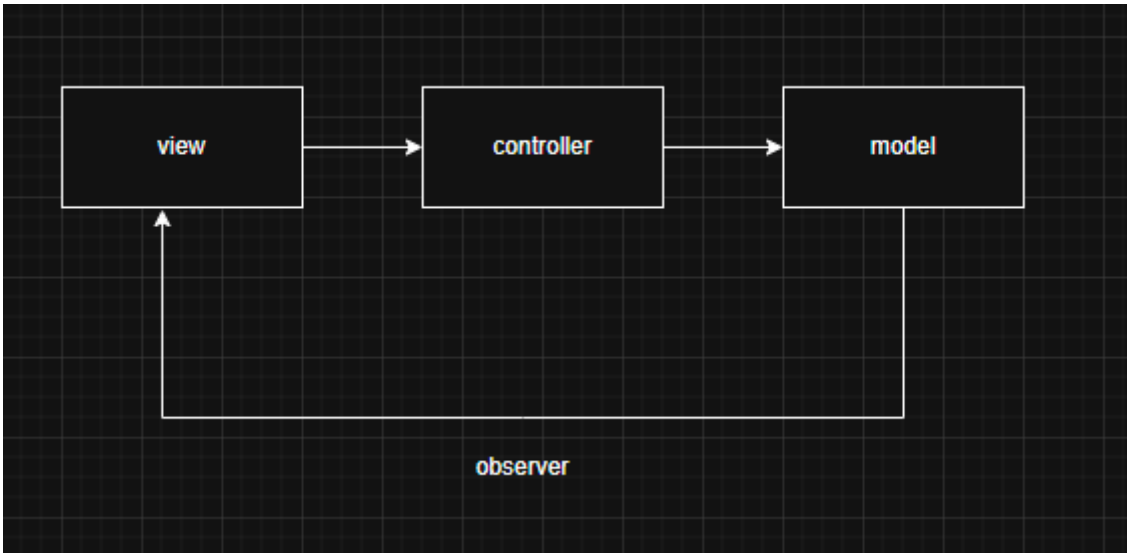
3.1 Architecture MVC

L'application est organisée selon une architecture proche du modèle MVC (Model – View – Controller). Cette séparation permet de répartir les responsabilités entre les différentes parties de l'application.

- Le Model contient les données du jeu ainsi que les classes représentant les MaGos et les messages ;
- La View est responsable de l'affichage de l'interface graphique et des interactions avec l'utilisateur ;
- Les Controllers assurent la communication entre la vue et le modèle en exécutant les différentes actions de l'utilisateur.

Cette architecture améliore la lisibilité du projet, facilite les évolutions futures et limite le couplage entre les différentes parties de l'application.

Figure 1 : Diagramme simplifié de l'architecture MVC



Comme on peut le voir dans ce diagramme, le contrôleur reçoit les actions de l'utilisateur, modifie le modèle lorsque cela est nécessaire puis la vue est automatiquement mise à jour grâce au pattern Observer.

3.2 Pattern Singleton

Le gestionnaire principal du jeu (GameManager) est implémenté sous la forme d'un Singleton.

Son rôle est de centraliser l'ensemble des composants principaux de l'application :

- Le modèle du jeu ;
- Le gestionnaire des messages ;
- La stratégie de sélection des messages ;
- Le timer.

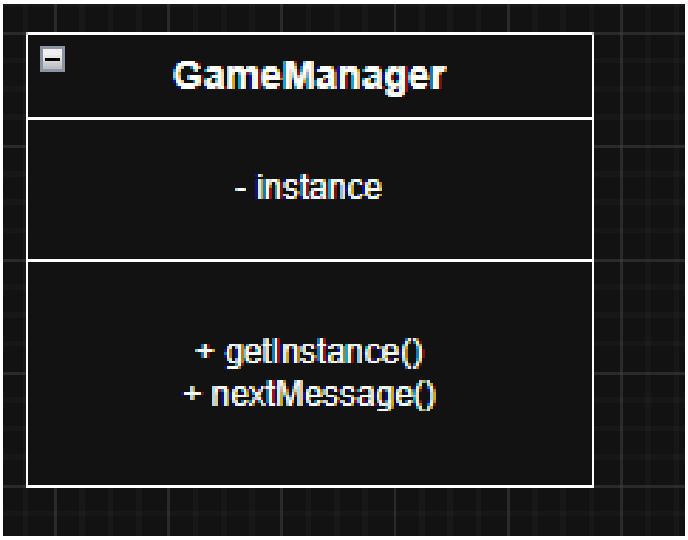
L'utilisation du Singleton garantit qu'une seule instance de ce gestionnaire existe durant toute l'exécution de l'application. Cela évite des incohérences qui pourraient apparaître si plusieurs parties du programme manipulaient plusieurs gestionnaires différents.

L'accès à cette instance unique est réalisé grâce à la méthode :

‘GameManager.getInstance()’

Ce choix simplifie également la communication entre les différents contrôleurs.

Figure 2 : Diagramme du Singleton



3.3 Pattern Factory

La création des MaGos est réalisée à l'aide d'une fabrique (MaGoFactory).

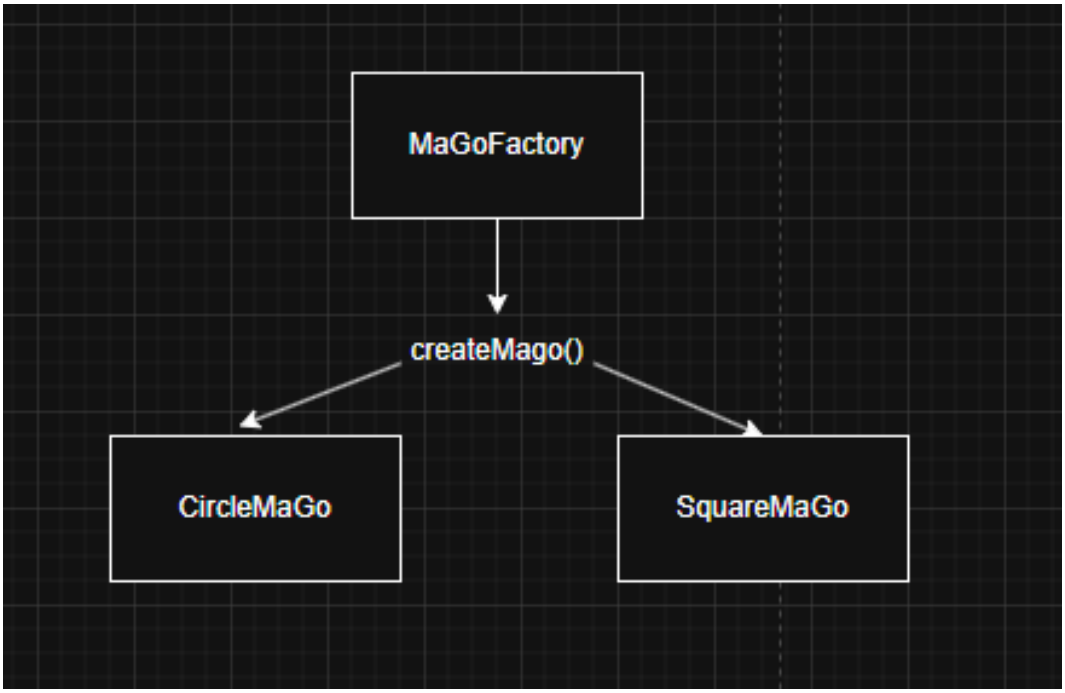
Cette classe est responsable de créer automatiquement une population composée de MaGos Circle et Square.

La décision concernant le type de MaGo est entièrement centralisée dans cette classe.

Ainsi, le reste du programme ne connaît pas les différentes classes concrètes (CircleMaGo ou SquareMaGo) et manipule uniquement des objets de type MaGo.

Cette approche respecte le principe de séparation des responsabilités et facilite l'ajout de nouveaux types de MaGos dans le futur.

Figure 3 : Diagramme UML de la Factory



3.4 Pattern Strategy

La sélection du prochain message repose sur le pattern Strategy.

Une interface commune (MessageSelectionStrategy) définit une méthode permettant de choisir un message.

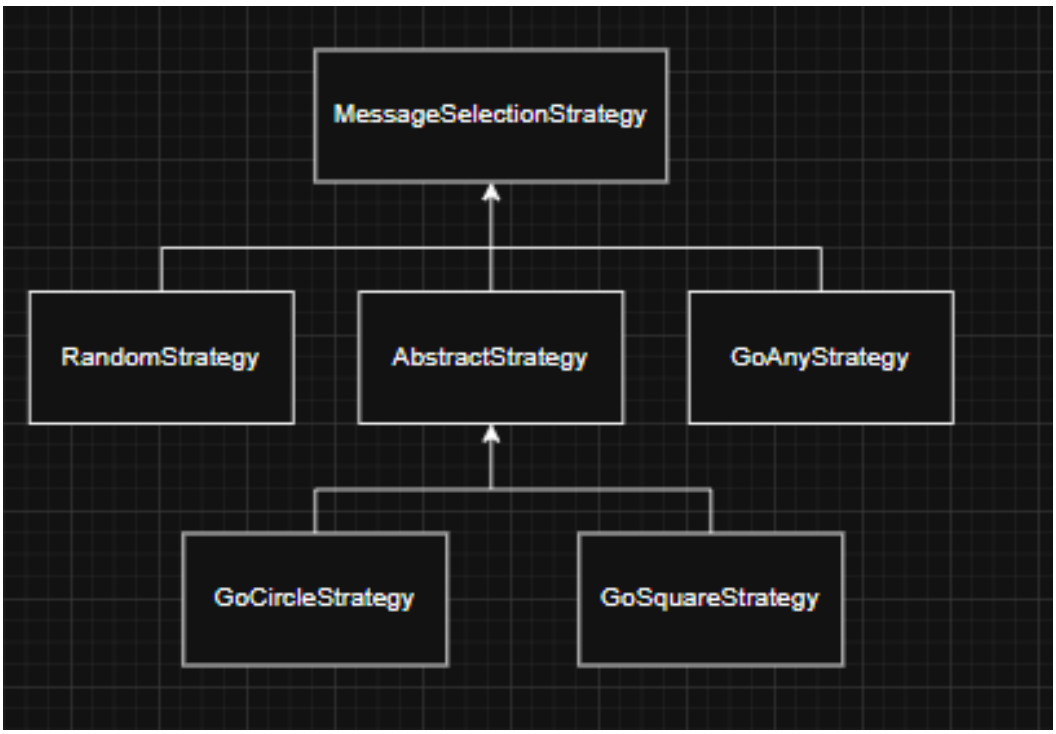
Chaque stratégie implémente un comportement différent :

- RandomStrategy choisit un message aléatoirement ;
- GoCircleStrategy sélectionne le message optimisant les MaGos Circle ;
- GoSquareStrategy sélectionne le message optimisant les MaGos Square ;
- GoAnyStrategy choisit aléatoirement un type présent dans la population puis applique la stratégie correspondante.

Grâce à cette organisation, le comportement de sélection peut être modifié dynamiquement pendant l'exécution sans modifier le reste du programme.

Le GameManager ne dépend donc pas d'une stratégie particulière mais uniquement de l'interface commune.

Figure 4 : Diagramme du pattern Strategy



3.5 Pattern Observer

La mise à jour automatique de l'interface graphique repose sur le pattern Observer.

Le modèle (GameModel) implémente l'interface Subject.

La vue principale (MainView) implémente l'interface Observer.

À chaque modification du modèle (nouveau message, changement du bonheur d'un MaGo, modification manuelle du statut…), le modèle appelle :

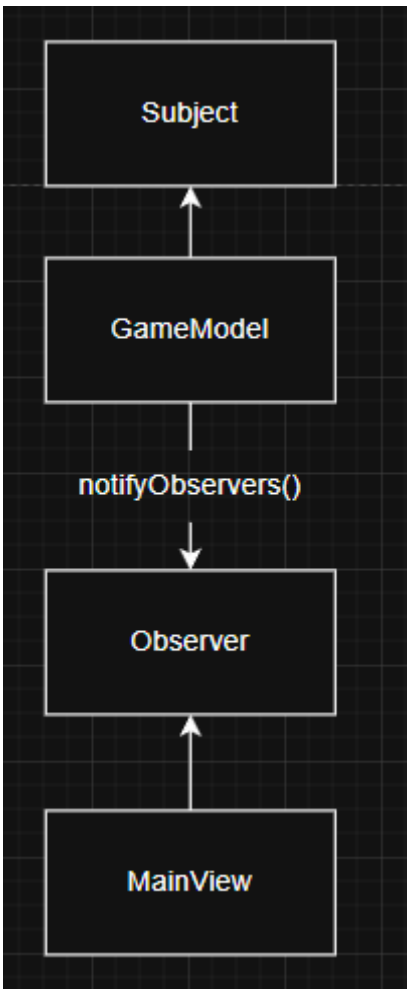
« notifyObservers(); »

La vue reçoit alors automatiquement la notification grâce à sa méthode :

« update(); »

Cette approche évite que le modèle connaisse directement la vue et réduit fortement le couplage entre les différentes couches de l'application.

Figure 5 : Diagramme Observer



3.6 Justification des choix

Les différents patterns utilisés répondent chacun à un besoin précis.

- Le Singleton garantit l'existence d'un unique gestionnaire de jeu.
- La Factory centralise la création des différents MaGos.
- Le pattern Strategy permet de modifier facilement le comportement de sélection des messages sans modifier les autres classes.
- Enfin, le pattern Observer assure une mise à jour automatique de l'interface graphique tout en conservant une faible dépendance entre le modèle et la vue.

L'utilisation combinée de ces différents patterns permet d'obtenir une architecture claire, modulaire et facilement extensible.

4. Limitations

Malgré le respect des fonctionnalités demandées dans l'énoncé, certaines limitations subsistent dans l'application. Elles ne remettent pas en cause son fonctionnement mais pourraient être améliorées dans une version future.

4.1 Gestion des messages

Les messages sont chargés une seule fois au démarrage de l'application à partir d'un fichier texte. Si ce fichier est modifié pendant l'exécution du programme, les changements ne sont pas pris en compte.

Une amélioration possible serait de permettre un rechargement dynamique des messages ou l'ajout de nouveaux messages directement depuis l'interface graphique.

4.2 Nombre de types de MaGos

L'application gère actuellement uniquement deux types de MaGos :

- Circle
- Square

Grâce à l'utilisation du polymorphisme et du pattern Strategy, l'ajout d'un nouveau type resterait relativement simple. Il suffirait de créer une nouvelle sous-classe de MaGo ainsi qu'une stratégie de sélection adaptée si nécessaire.

4.3 Gestion des erreurs

Le projet suppose que le fichier contenant les messages est correctement présent dans les ressources de l'application.

Si ce fichier est absent ou inaccessible, une exception est levée afin de signaler clairement le problème. Une version plus évoluée pourrait proposer un message d'erreur graphique permettant à l'utilisateur de comprendre immédiatement l'origine du problème.

5. Conclusion

Ce projet m'a permis de mettre en pratique les principaux concepts de la programmation orientée objet étudiés durant le cours. La réalisation de l'application HELBMaGo m'a amené à concevoir une architecture claire en séparant les différentes responsabilités grâce au modèle MVC et en appliquant plusieurs design patterns tels que Singleton, Factory, Strategy et Observer.

Au-delà des fonctionnalités demandées, ce projet m'a également permis d'améliorer la qualité générale de mon code. Une attention particulière a été portée à la lisibilité en utilisant des commentaires Javadoc, en centralisant les constantes dans une classe dédiée et en limitant les duplications de code lorsque cela était possible. Des tests unitaires ont également été réalisés afin de vérifier le bon fonctionnement de certaines fonctionnalités importantes.

Même si certaines améliorations pourraient encore être envisagées, les objectifs fixés par l'énoncé ont été atteints. L'application permet de gérer une population de MaGos, de sélectionner les messages selon différentes stratégies, de faire évoluer leur niveau de bonheur et de mettre automatiquement à jour l'interface graphique.

Ce projet constitue une bonne synthèse des notions abordées durant le cours et m'a permis de mieux comprendre l'intérêt d'une architecture logicielle bien organisée ainsi

que l'apport des design patterns dans le développement d'une application orientée objet.

6. Accès LLM

Email : azarkan.ilyas.java2526@gmail.com

Mot de passe : Ladiqekuj.2006

Lien des 5 dernieres conversations ChatGPT :

https://chatgpt.com/share/6a5ba432-a9b4-83ed-b33e-ed216e2af800
https://chatgpt.com/share/6a5ba43c-6ef4-83eb-a7b3-ab011808e068
https://chatgpt.com/share/6a5ba449-ce88-83eb-8776-7bc7a1dc555e
https://chatgpt.com/share/6a5ba45c-a35c-83eb-acbc-b8e77eaf6f8e
https://chatgpt.com/share/6a5ba46f-8d34-83eb-aced-07302d74adfe